भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

CS6903: Network Security
(Module 5: HTTPS: HTTP Ovel TLS)

Saurabh Kumar
CSE Department, IIT Hyderabad

Acknowledgement

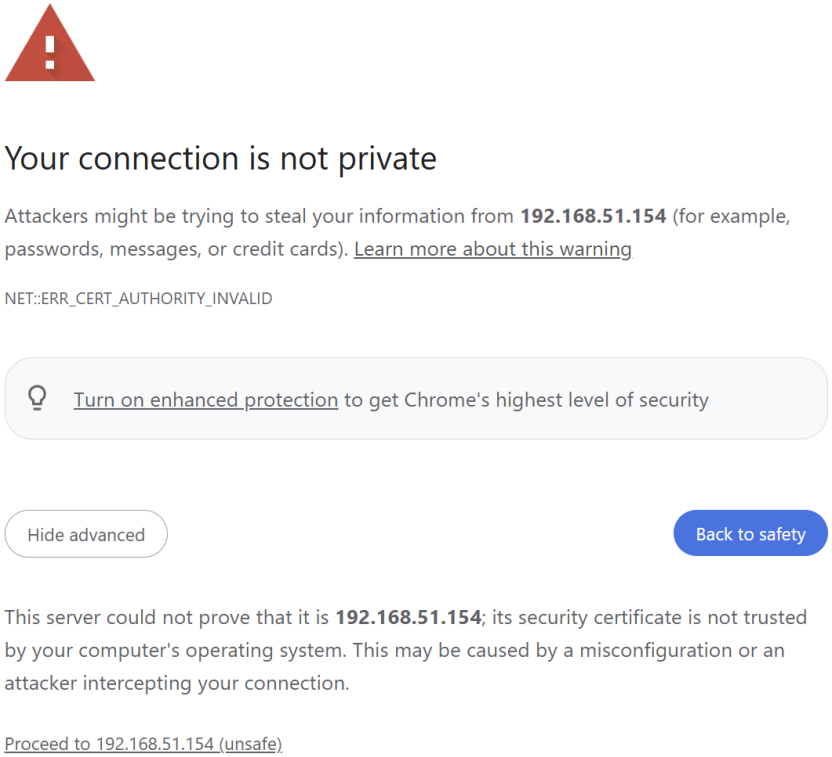
- Instructors from previous course floating
  - Prof. Bheemarjuna Reddy Tamma, Prof. Kotaro Kataoka, Prof. Antony Franklin
- Professors from other Universities/Insitutes
  - Sandeep K. Shukla, Stefan Savage, Dan Boneh, Kirill Levchenko, Alex Gantman, Deian Stefan, Nadia Heninger, Alex Dent, Vitaly Shamtikov, Robert Turner
- Other internet sources

HTTPS: HTTP Over TLS

- HTTPS: RFC2818
- User https://url rather than http://url
  - And port 443 rather than 80
- Connection initiation
  - TCP handshake, TLS handshake, and then HTTP request(s)
- Encrypts
  - URL, document contents, form data, cookies, HTTP headers
- Connection Close
  - **"Connection: close"** in HTTP record
  - TLS level exchange **close_notify** alerts
  - Can then close TCP connection using **FIN and Ack** messages

Other Problems with SSL/TLS

- Browser provides feedback to user about whether HTTPS is in use, but many users don't pay attention
- If a certificate is bad/unknown, browser issues warning dialogs
  - certificate_expired
  - certificate_revoked
  - bad_certificate
  - unknown_ca
  - bad_record_mac

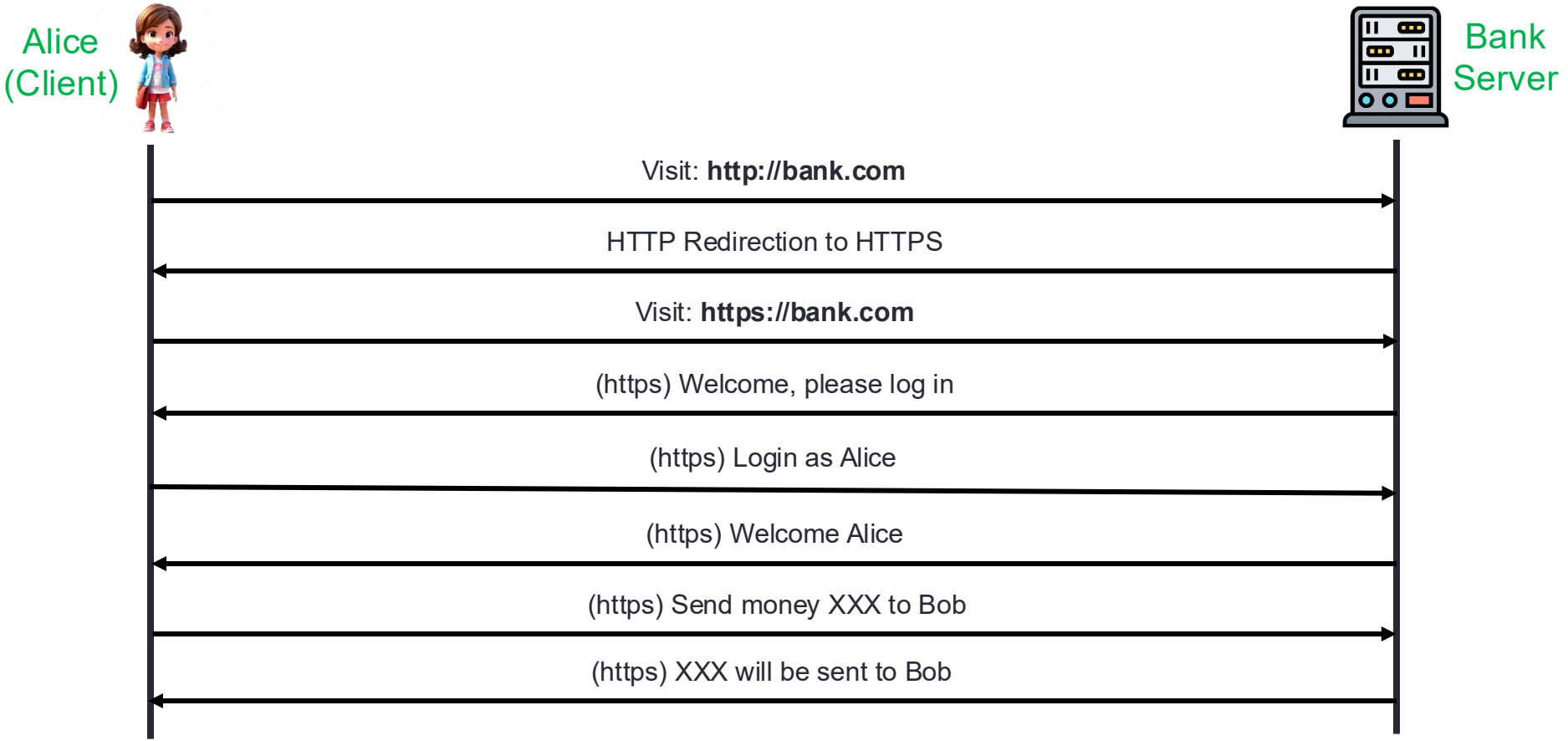


https://badssl.com/dashboard/

Other Problems with SSL/TLS

- Legacy use pattern: user browses site with HTTP, upgrades to HTTPS for checkout or login
- "Just in time" HTTPS:
  - Login page displayed with HTTP, but Form posted with HTTPS hopefully!
  - Appears secure but it may not:
    - MITM attacker can corrupt HTTP login page during transit from Web Server to Client
    - SSL stripping during form post: nothing indicates that the actual connection didn't use SSL
      - Send userID/password somewhere else during form post

Login Form Over HTTPS

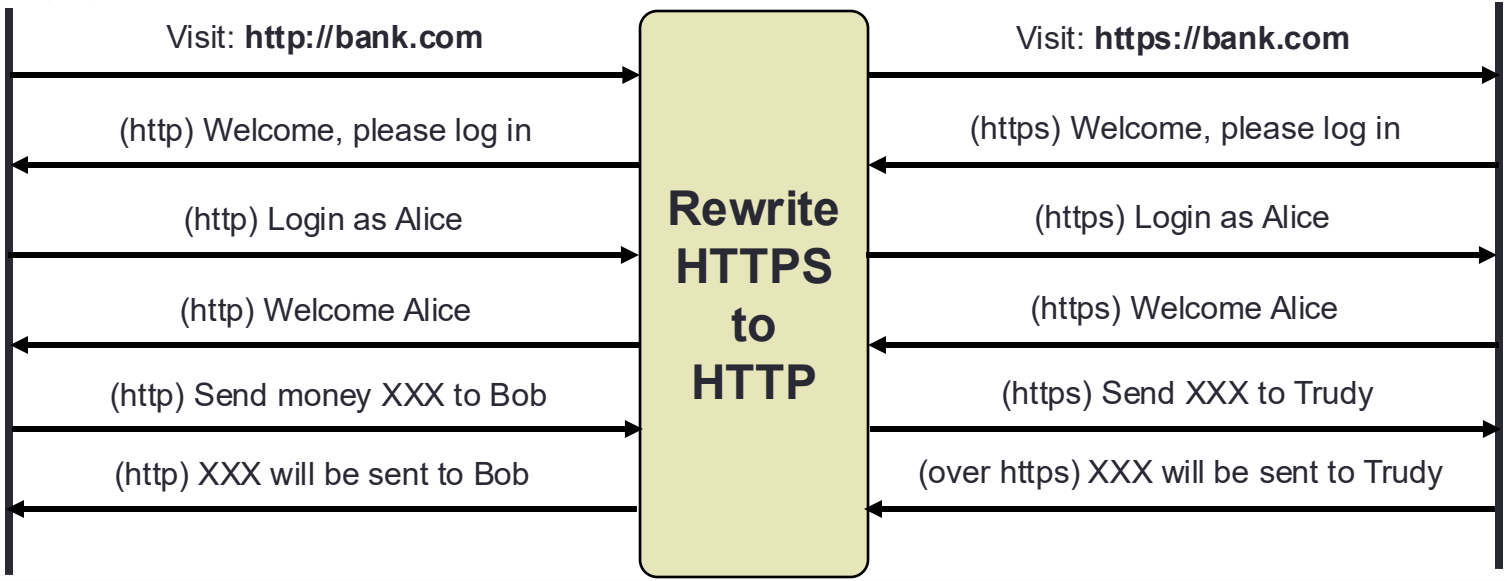


Stripping HTTPS from Login Form

Alice (Client)

Trudy (Attacker)

Bank Server

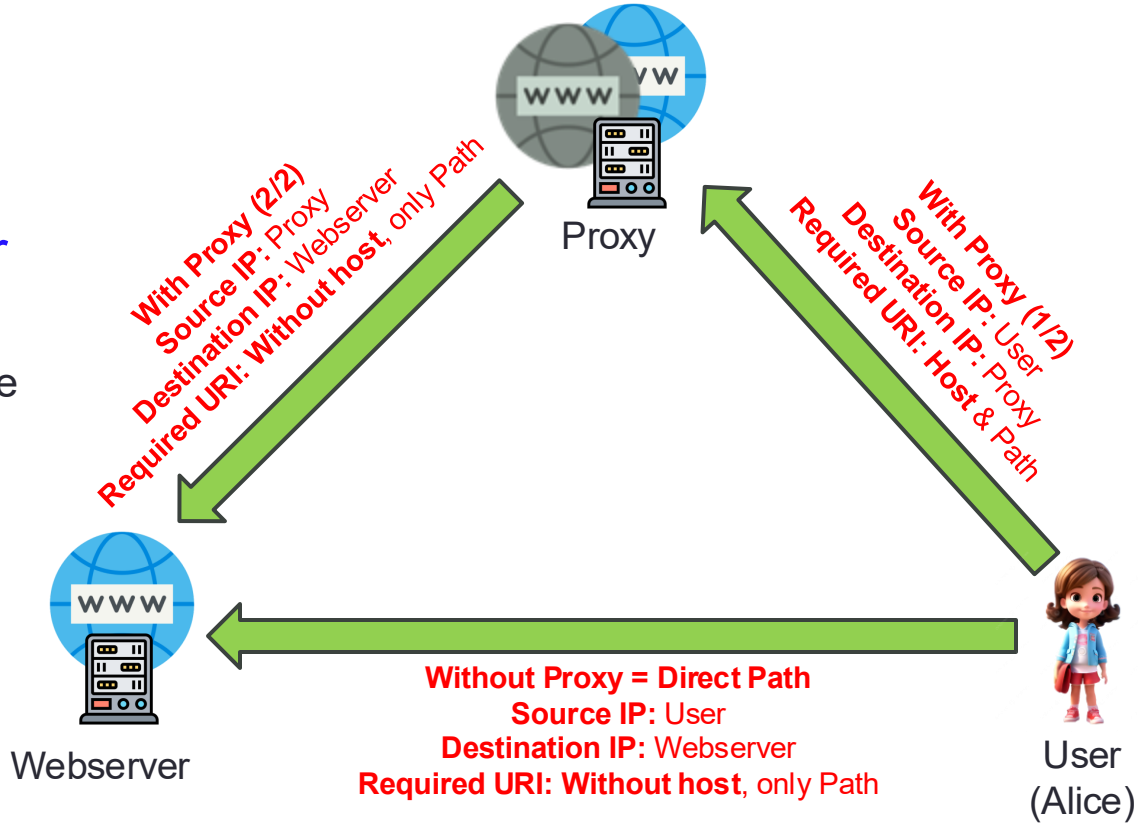


Solution: Before returning HTML for login page, browser checks for HTTPS; or if page fetched via HTTP, redirect to the HTTPS version

Integrating TLS with HTTP: HTTPS

Two complications

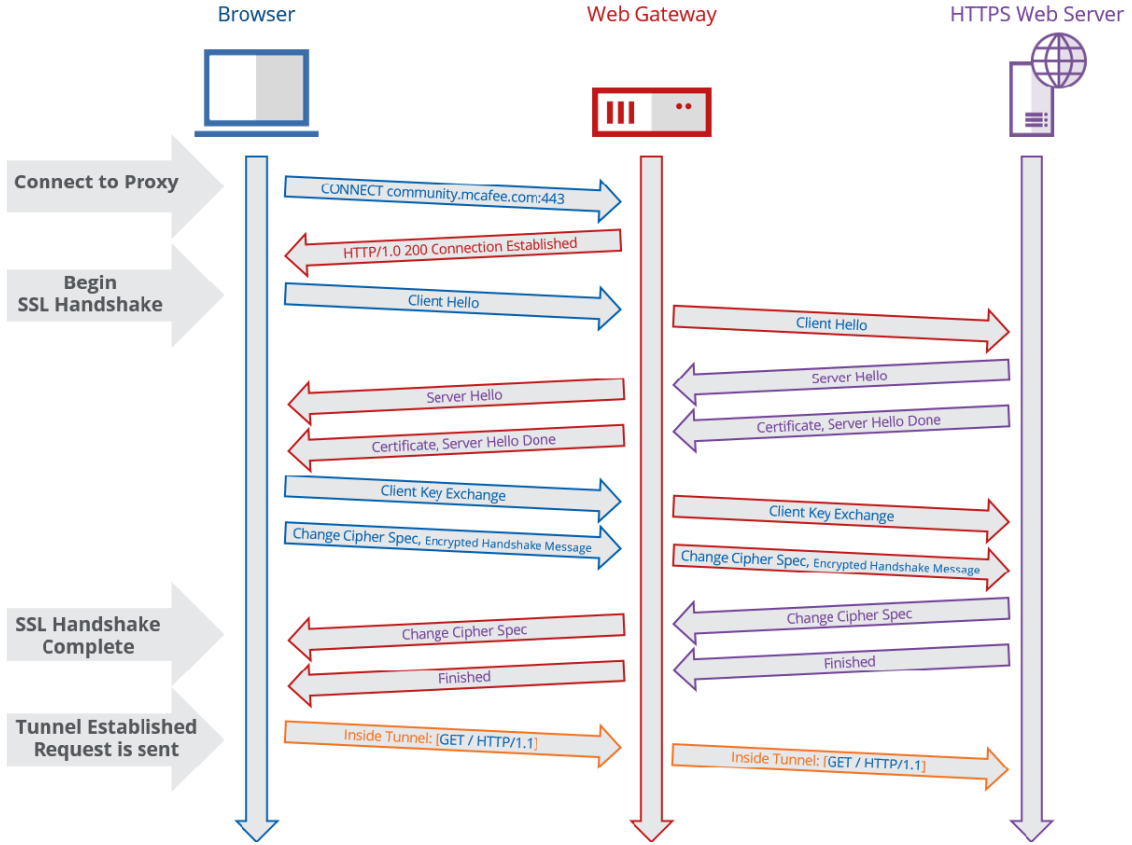
- Web proxies/GWs: MITM
  - Pass-through (bypass or forwarding) web proxy for HTTPS traffic
    - Caches resources to reduce bandwidth and achieve speed up & hides your IP address while browsing



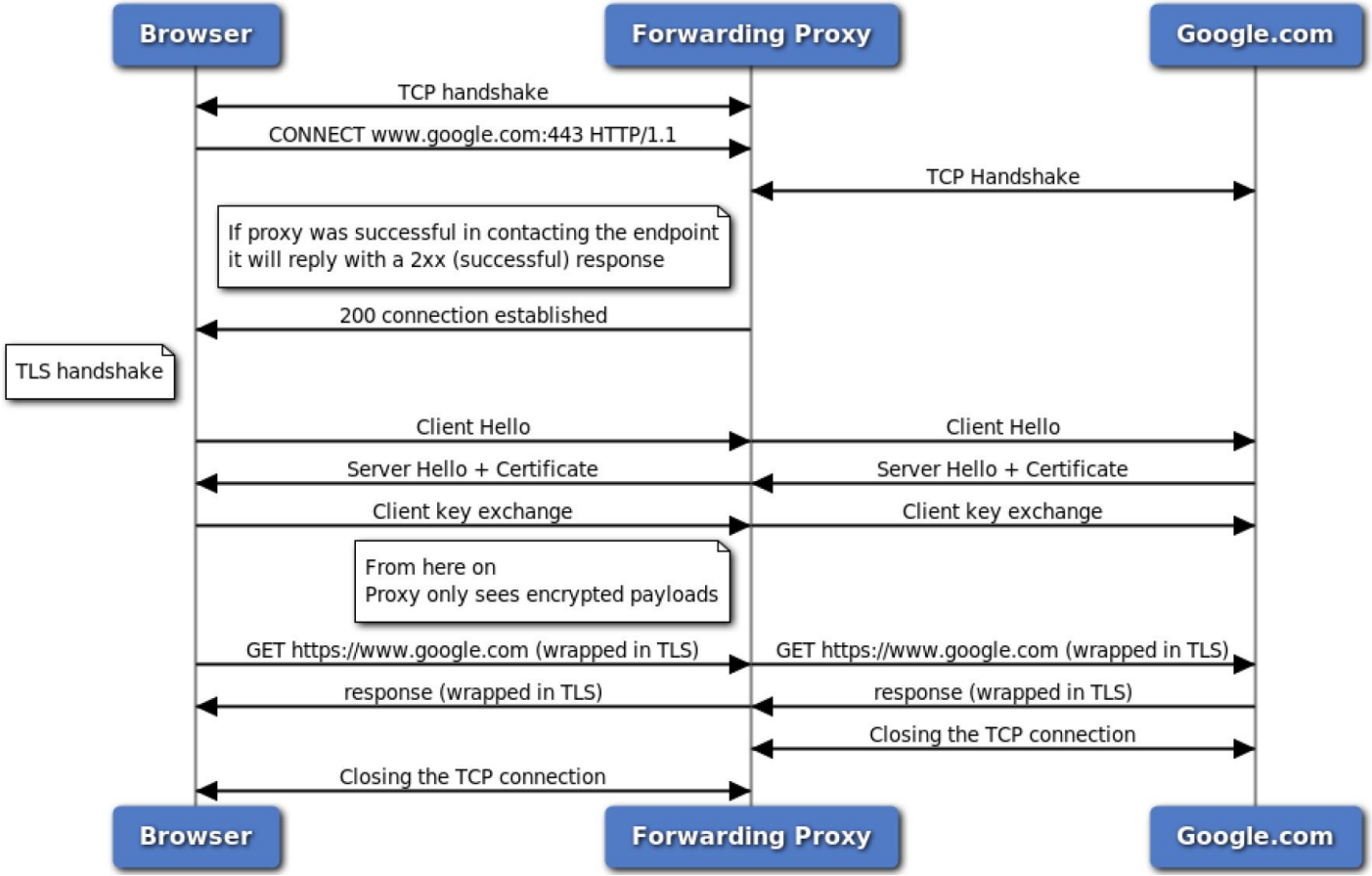
Integrating TLS with HTTP: HTTPS

Two complications

- Web proxies/GWs: MITM
  - Pass-through (bypass or forwarding) web proxy for HTTPS traffic
    - HTTP **CONNECT** method before TLS client_hello to seek Proxy to create a TLS tunnel between Client and Server



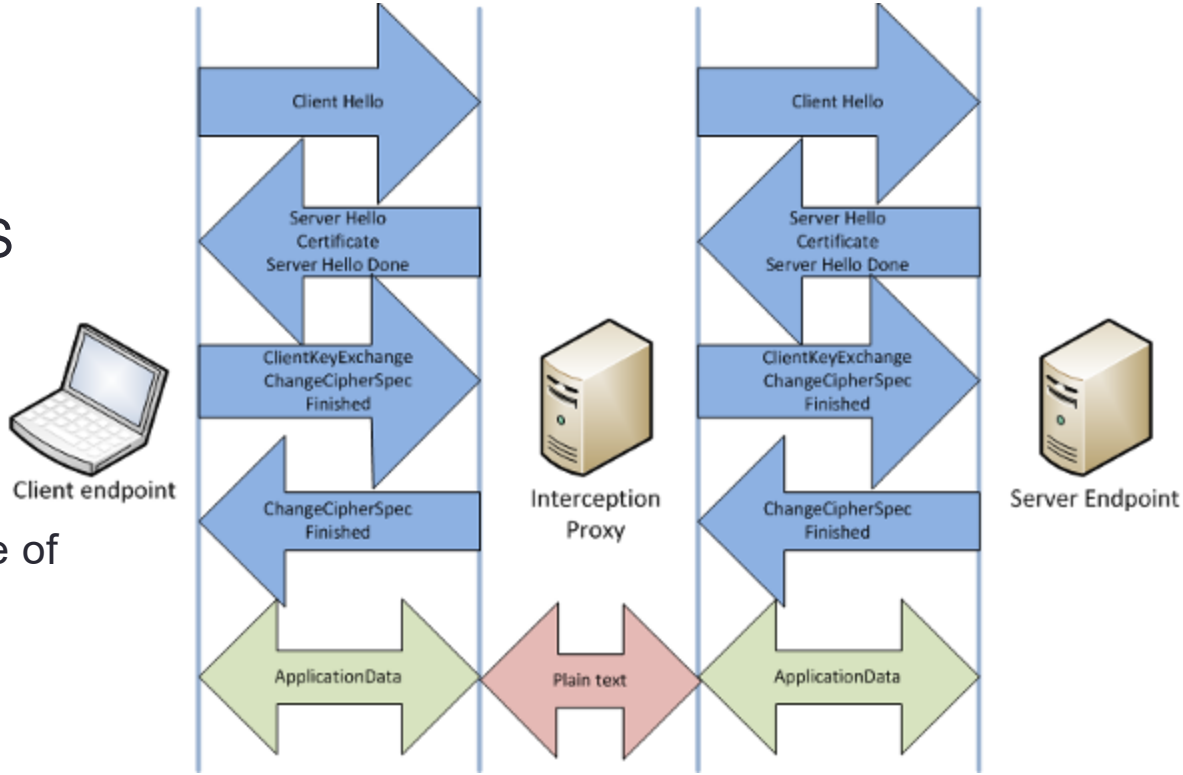
Access Google.com with Forwarding Proxy



Integrating TLS with HTTP: HTTPS

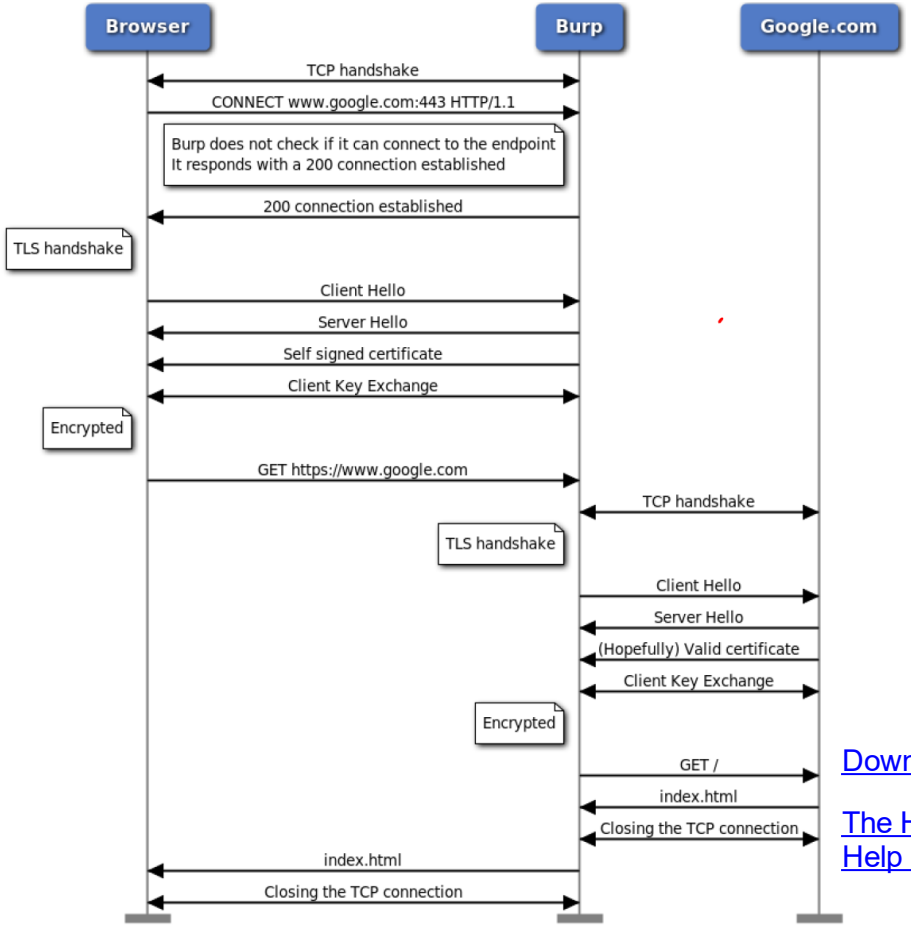
Two complications

- Web proxies/GWs: MITM
  - Bypass Proxy for HTTPS
  - Intercept HTTPS Traffic
    - Squid
    - Cert of Proxy need to be stored in Trusted Root Certificate Authorities store of Client's browser



https://blog.scphillips.com/posts/2017/04/intercept-https/

Access Google.com Using Burp Suit



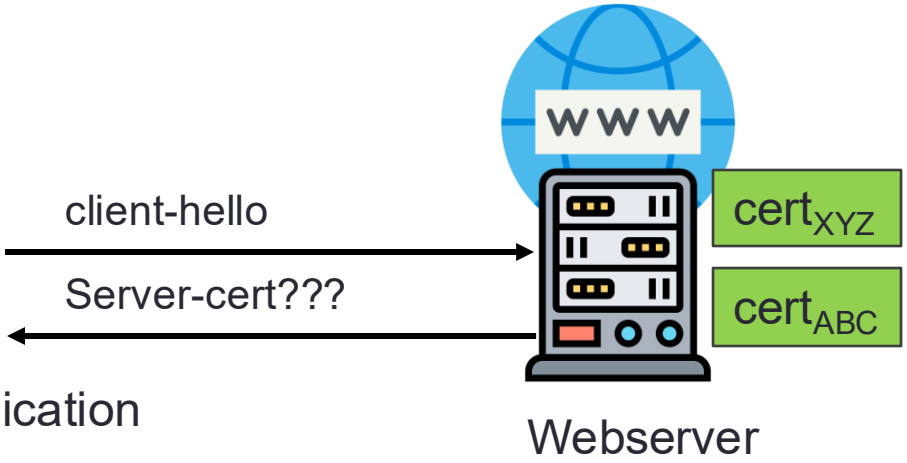
[Download Burp Suite Community Edition – PortSwigger]

[The HTTPS interception dilemma: Pros and cons - Help Net Security]

Integrating TLS with HTTP: HTTPS

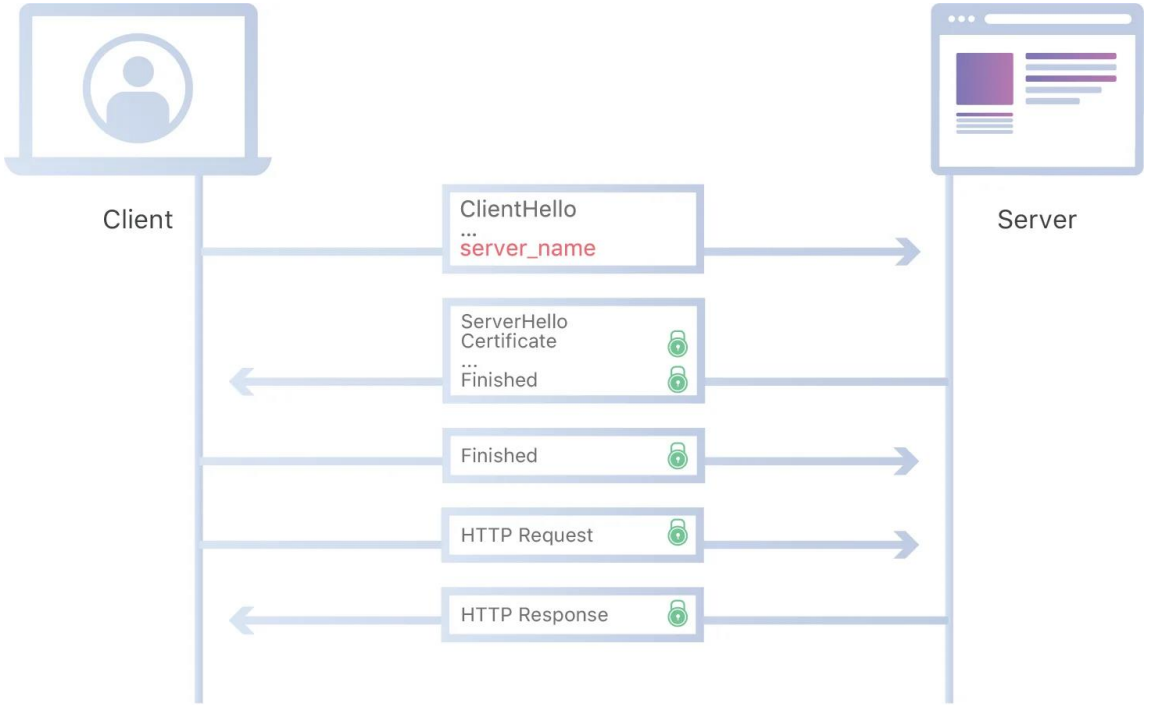
Two complications

- Web proxies/GWs: MITM
- Virtual Hosting
  - Two sites at the same IP address
  - Solution in TLS 1.1: Server Name Indication
    - Client_hello_extension: server_name=xyz.com
  - Implemented since FireFox2 and IE7 (vista)



https://www.ssl.com/article/sni-virtual-hosting-for-https/

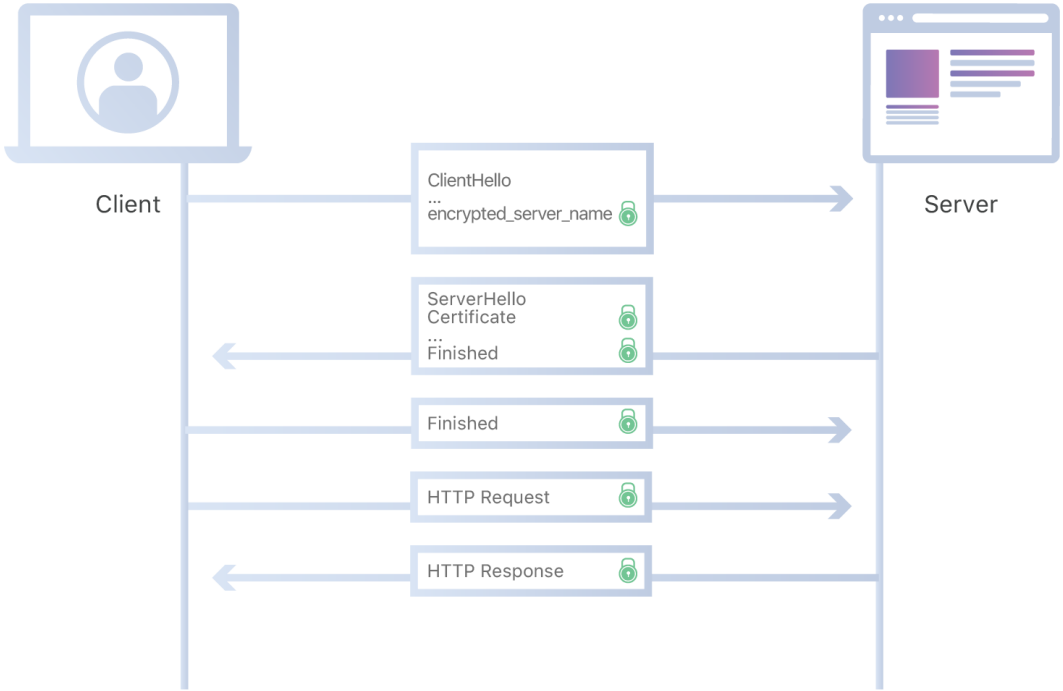
TLS with Unencrypted SNI



CLOUDFLARE

ISPs and firewalls track which sites a user is visiting!

TLS 1.3 Offers Encrypted SNI



CLOUDFLARE

Where does the encryption key come from?

Encrypted SNI Extension

- Where does the encryption key come from if client and web server haven't negotiated one yet?
  - DNS: Web server publishes a public key of DHE from its end on a well-known DNS record
  - Client making DNS query gets public key as well & generates a shared encryption key for encrypting SNI in client_hello
  - Web server also generates the same encryption key after receiving public key of DHE from client side
  - Note: These DHE paras are different from ones used for key exchange
- Simply using DNS (which is, by default, unencrypted) would make Encrypted SNI extension worthless
  - Solution:
    - DNS over TLS or DNS over HTTPS
    - Plus DNSSEC (offers signed DNS responses)

Why is HTTPS Not Used For All Web Traffic

- Crypto slows down web servers (but not by much if done right)
- Some ad-networks still do not support HTTPS
  - Reduced revenue for publishers
- Incompatible with virtual hosting (older browsers)
  - March 2017: IE6 ≈ 1-5% in China(ie6countdown.com)
- Aug 2014: Google boosted ranking of sites supporting HTTPS!

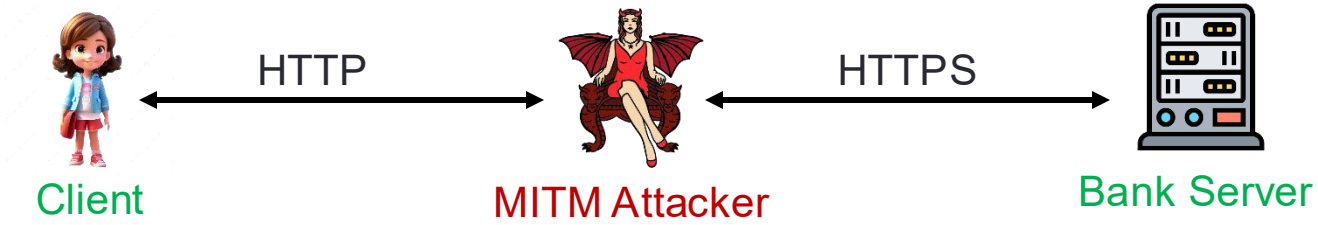
Problems With HTTPS and Defences

Problems

- Stripping of upgrade from HTTP to HTTPS
- Mixed content: HTTP and HTTPS on the same page
- Forged Certificates
- Peeking Through SSL: Traffic Analysis?

HTTP ➔ HTTPS Upgrade

- Legacy use pattern:
  - Browser site over HTTP; move/upgrade to HTTPS for checkout
  - Connect to back over HTTP; upgrade to HTTPS for login
- SSL strip attack: prevent the upgrade



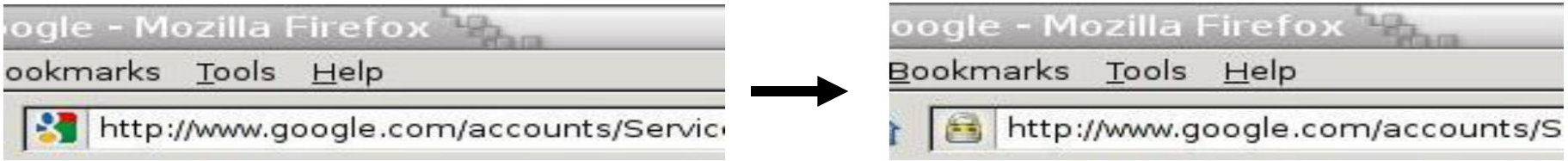
<a href="https://..... > → <a href="http://..... > before forwarding to client

Location: https://..... → Location: http://.....

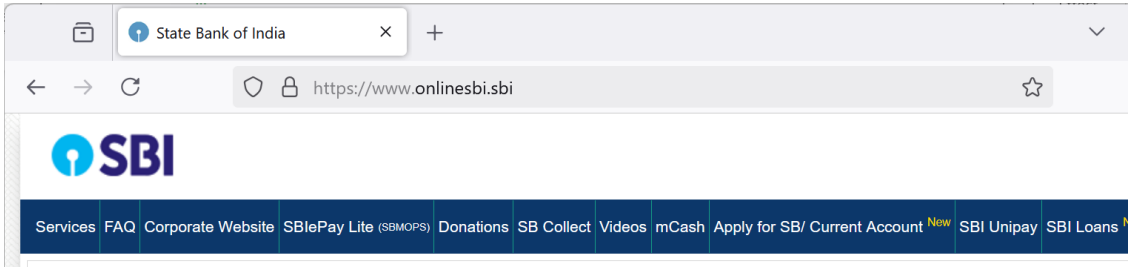
<form action=https://…. > → <form action=http://…. >

Tricks and Details

Tricks: Drop-in a clever favicon icon (old browsers)

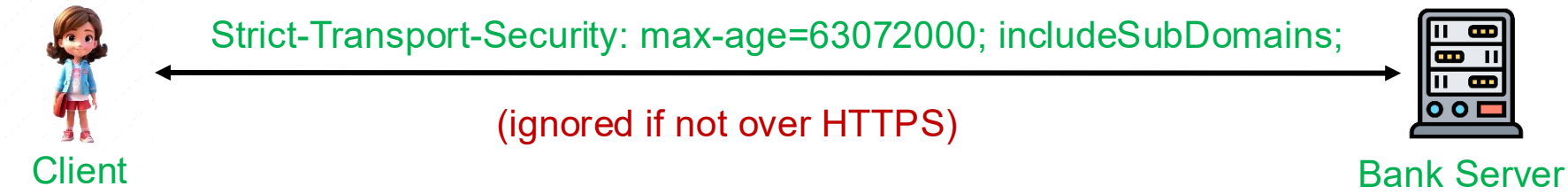


➔favicon icon no longer presented in address bar



Always on SSL: Add 301 redirects to HTTPS URLs by rewriting HTTP (port 80) requests to HTTPS (port 443) on your webserver

Defence: HTTP Strict Transport Security (HSTS)



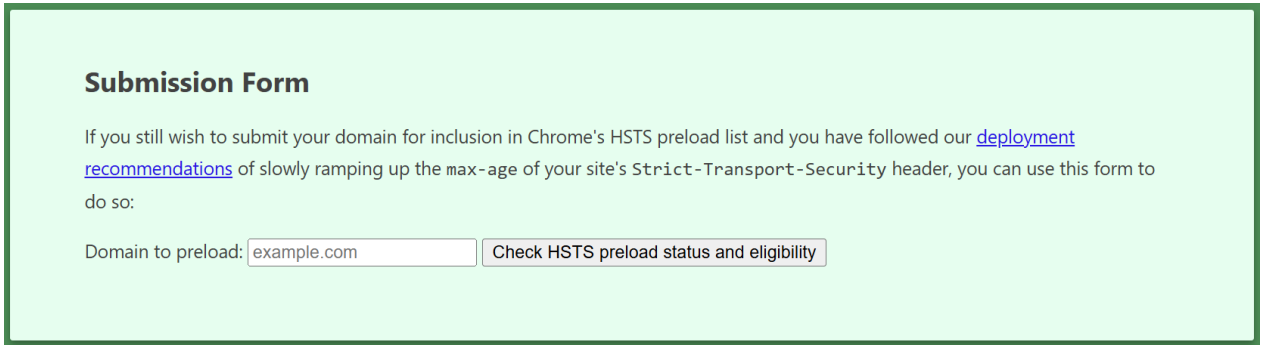
- Header tells browser to connect over HTTPS to the server for max-age
- So, subsequent visits must be over HTTPS
  - Browser refuses to connect over HTTP if site present an invalid certificate
  - Entire site and its subdomains must be served over valid HTTPS
- Note: HSTS flag deleted when user “clears private data”

chrome://net-internals/#hsts and https://www.chromium.org/hsts

Preloaded HSTS list in Browsers

Helps in enforcing HTTPS all the time

https://hstspreload.org/



Strict-Transport-Security: max-age=63072000; includeSubDomains; **preload**

HSTS is supported in Chrome, Firefox, Safari, Edge, Opere, etc.
Examples : Gmail, Paypal, Twitter, … (but not google.com)

Mixed Content: HTTP and HTTPS

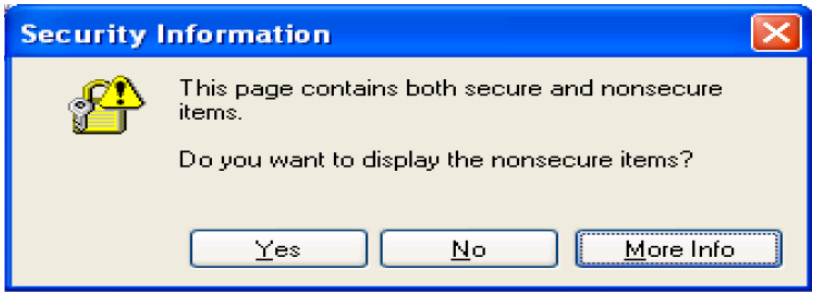
Page loads over HTTPS, but contains content over HTTP

(e.g. <script src="http://.../script.js>)

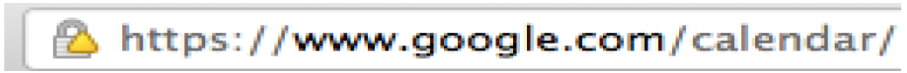
Never write this

➔ Active network attacker can hijack session by modifying script en-route to browser

IE7:

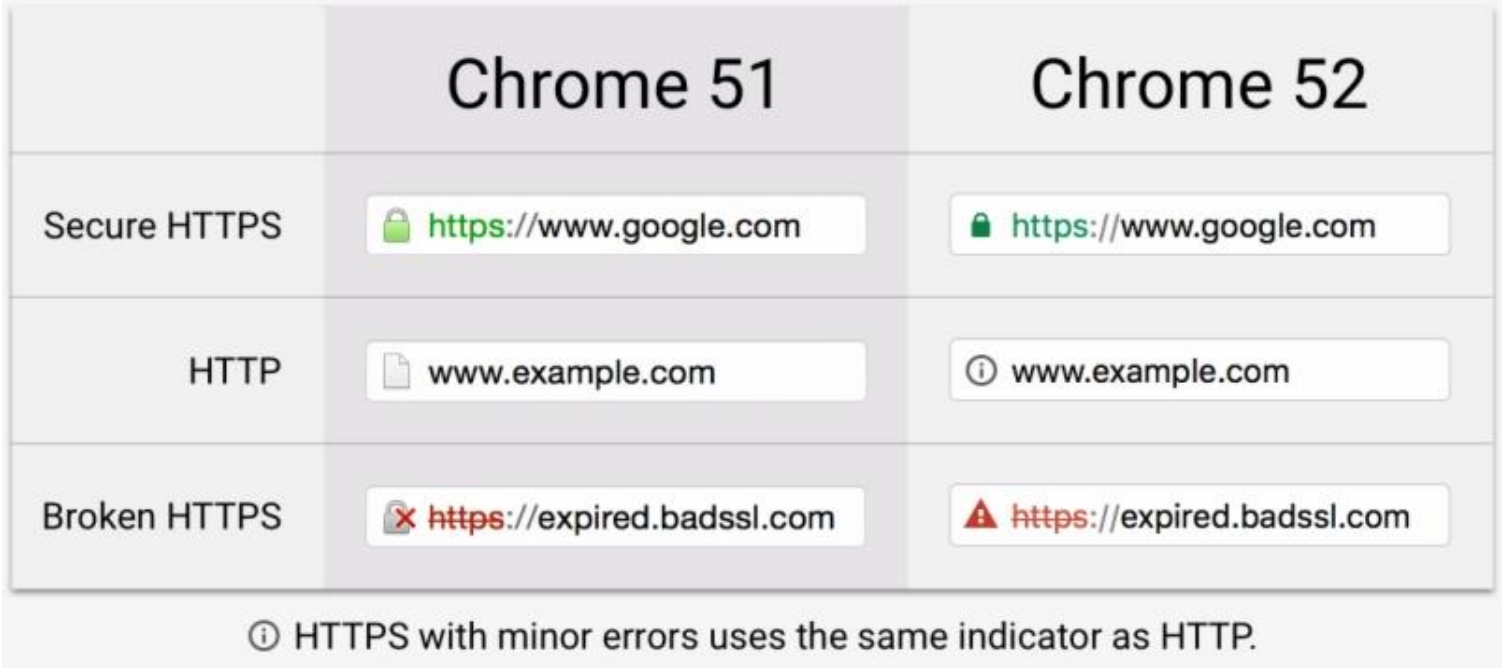


Old Chrome:



Mostly ignored by the user

Mixed Content: HTTP and HTTPS…



CSP: Upgrade Insecure Requests

- The problem: many pages use <img src="http://site.com/img">
- Cross Site Scripting (XSS) and data injection attacks
- Solution: gradual transition using Content Security Policy (CSP)

Content-Security-Policy: upgrade-insecure-requests A directive in HTML Response

```
<img src="http://site.com/img">
<img src="http://othersite.com/img">
<a href="http://site.com/img">
<a href="http://othersite.com/img">
```

```
<img src="https://site.com/img">
<img src="https://othersite.com/img">
<a href="https://site.com/img">
<a href="http://othersite.com/img">
```

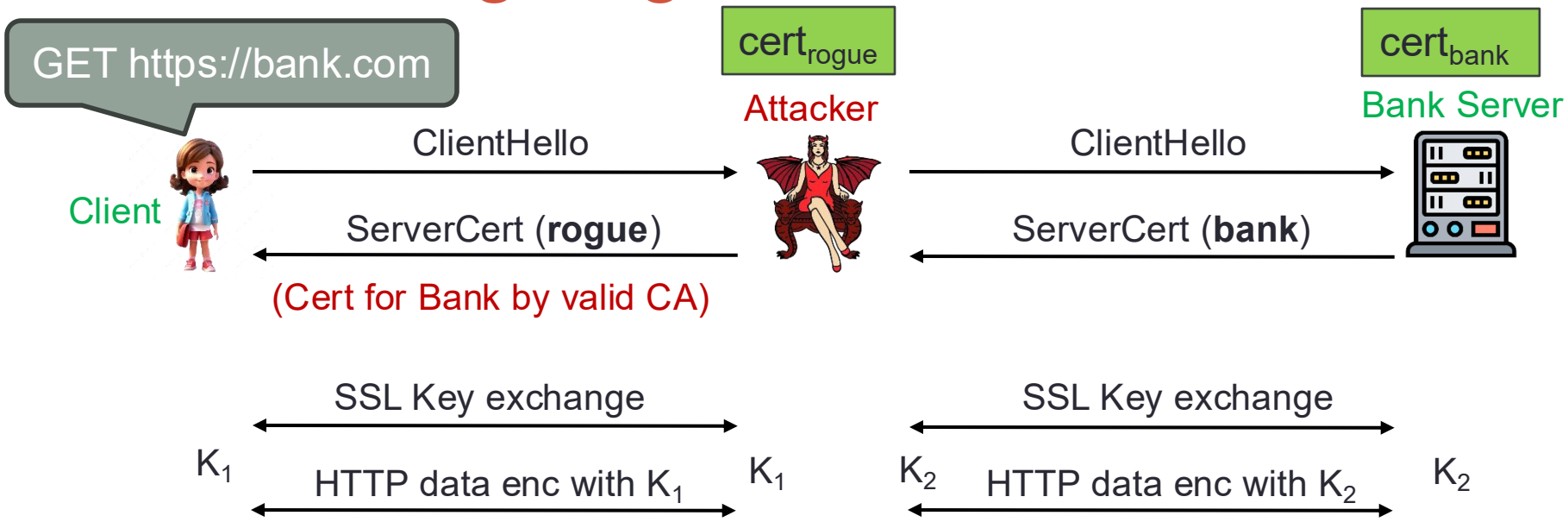
Always use protocol relative URLs <img src="//site.com/img"> & script-src 'self'

Forged Certificates: Wrong Issuance

- 2011: **Comodo** and **DigiNotar** CAs hacked, issued fake certs for Gmail, Yahoo!
- 2013: **TurkTrust** issued fake cert for gmail.com (discovered by pinning)
- 2014: **Indian NIC** (intermediate CA trusted by the root CA **IndiaCCA**) issued certs for Google and Yahoo! Domains
  - India CCA revoked NIC's intermediate certificate
  - Chrome restricts India CCA root to only seven Indian domains
- 2015: **MCS** (intermediate CA cert issued by **CNNIC**) issued certs for Google domains
  - current CNNIC root no longer recognized by Chrome

➔ leads to passive & active MITM attacks without a warning on user's session

MITM Using Rogue Certificate



- Attacker hacks CA and creates a rogue cert of Bank to launch MITM attacks.
- Attacker proxies data between user and bank.
- Sees all traffic and can modify data at will

What To Do??

1. Dynamic HTTP public-key pinning(HPKP) (RFC 7469)
   - Let a site declare CA that can sign its cert (similar to HSTS)
     - Web server tells a client which public key belongs to it
     - TOFU: Trust on First Use
   - On subsequent HTTPS, client rejects certs issued by other CAs

2. DNS CA Authorization (CAA) Records
   - Web server adds CAA record in its name server
   - CAs will query this record before cert issuance

Certificate Authority Authorization Record

thesslstore.com. CAA 0 issue “digicert.com”

thesslstore.com. CAA 0 issue wild “sectigo.com”

3. Certificate Transparency: [LL’12] (RFC 6962)
   - Idea: CA’s must advertise a log of all certs they issued
   - Browser will only use a cert if it is published on log server
     - Efficient implementation using Merklehash trees
     - Companies can scan logs to look for invalid issuance

HPKP Example (HTTP Header From Webserver)

Public-Key-Pins:
pin-sha256="d6qzRu9zOECb90Uez27xWltNsj0e1Md7GkYYkVoZWmM=";
pin-sha256="E9CZ9INDbd+2eRQozYqqbQ2yXLVKB9+xcprMF+44U1g=";
pin-sha256="LPJNul+wow4m6DsqxbninhsWHlwfp0JecwQzYpOLmCQ=";
max-age=2592000; includeSubDomains;
report-uri="http://example.com/pkp-report"

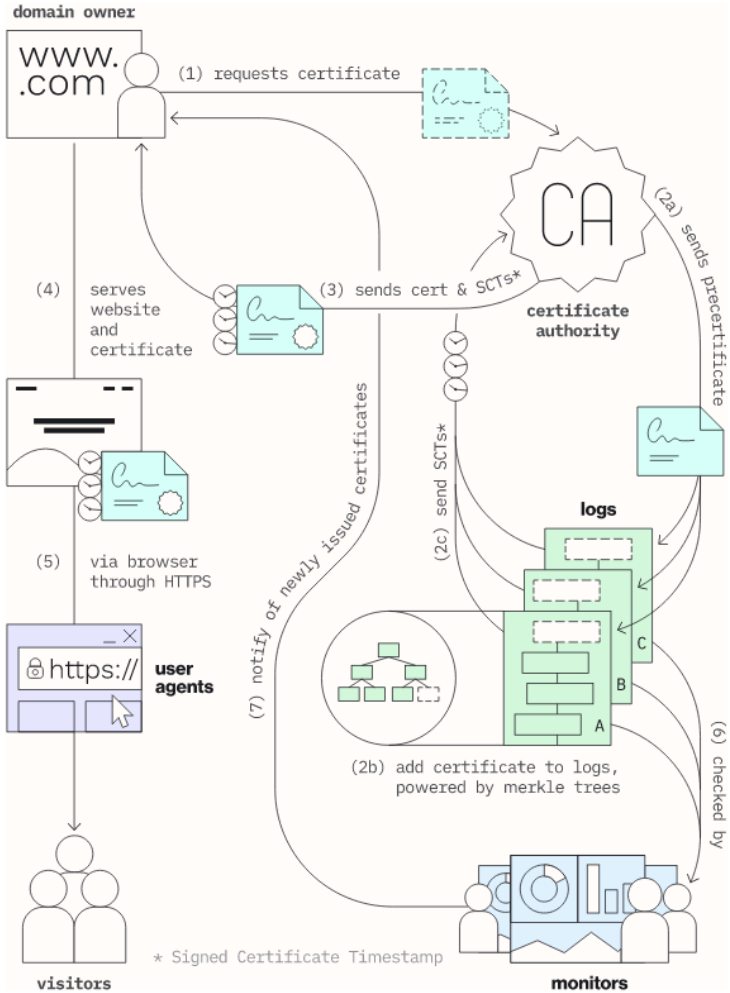
Max-age: 2,592,000 seconds is the most common max-age values used (30 days)

Examine browser's pinning DB: chrome://net-internals/#hsts

HPKP can be potentially very dangerous if configuration is done incorrectly

Certificate Transparency

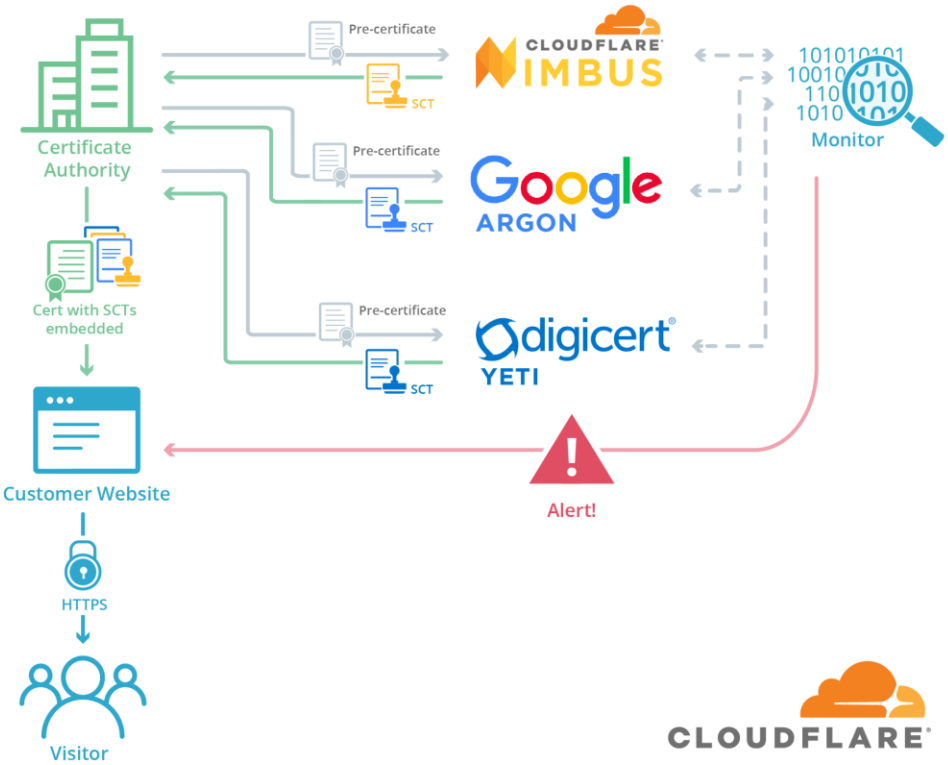
- An ecosystem that makes the issuance of Certs transparent and verifiable
- Certificates are deposited in public, transparent log servers by CAs
  - Logs are tamper-proof, append-only and publicly-auditable ledgers of certificates being created/updated/expired
- CA gets Signed Cert Timestamp (SCT) from the log server(s) and may embed into Cert issued
  - SCT: a promise to add Cert to the log within the Maximum Merge Delay (MMD)



How CT Works: Certificate Transparency

Certificate Transparency

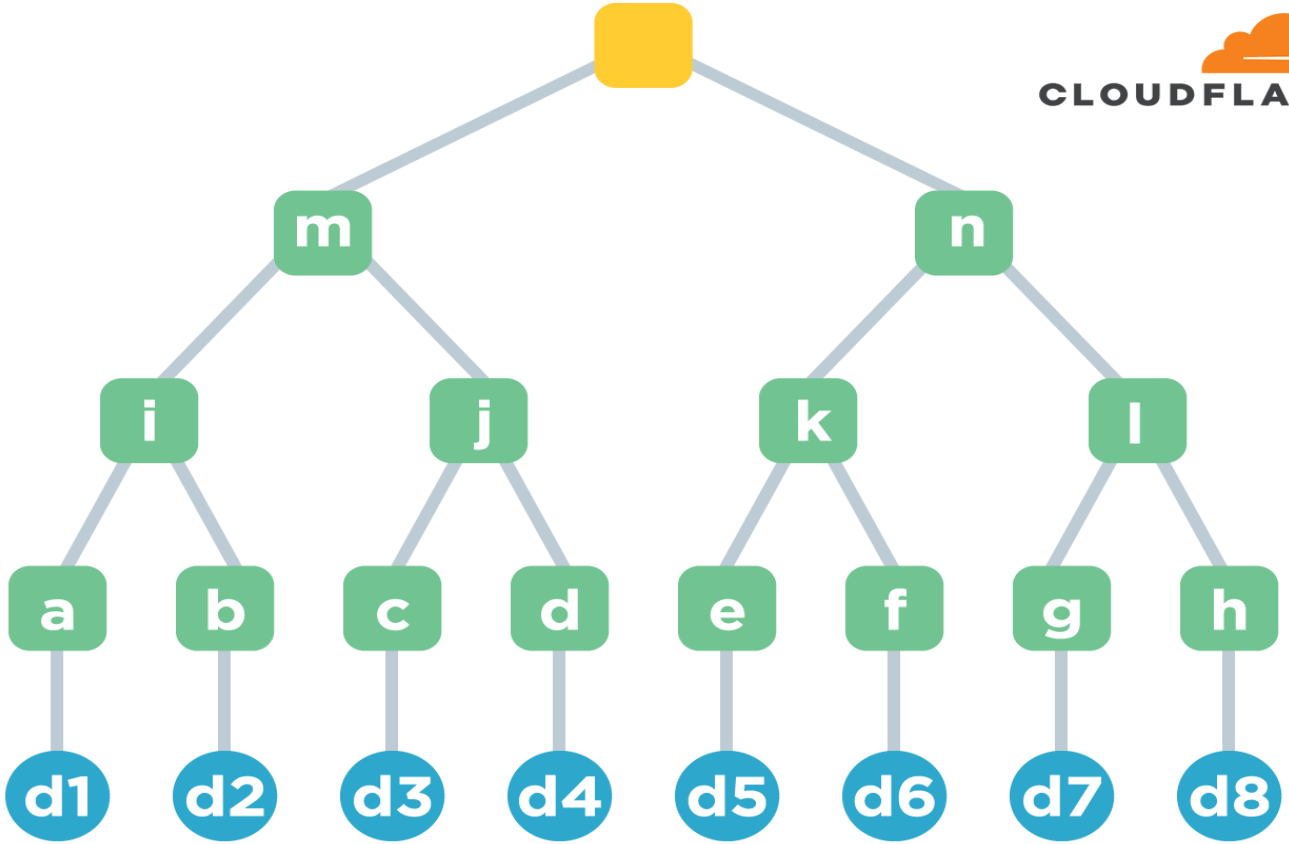
- Browsers can check for SCTs during TLS handshake and trust only the Certs logged in one or more vetted log servers
- Logs can be cryptographically monitored by anyone (Monitors)
  - Merkle Tree Hash to prevent tampering
  - It also offers nice balance to the log operators and auditors in terms of cost to add certs and validate their consistency
- CT only protects users if all certificates are logged!



CLOUDFLARE

https://blog.cloudflare.com/introducing-certificate-transparency-and-nimbus/

Certificate Transparency: Markel Hash Tree



https://blog.cloudflare.com/introducing-certificate-transparency-and-nimbus/

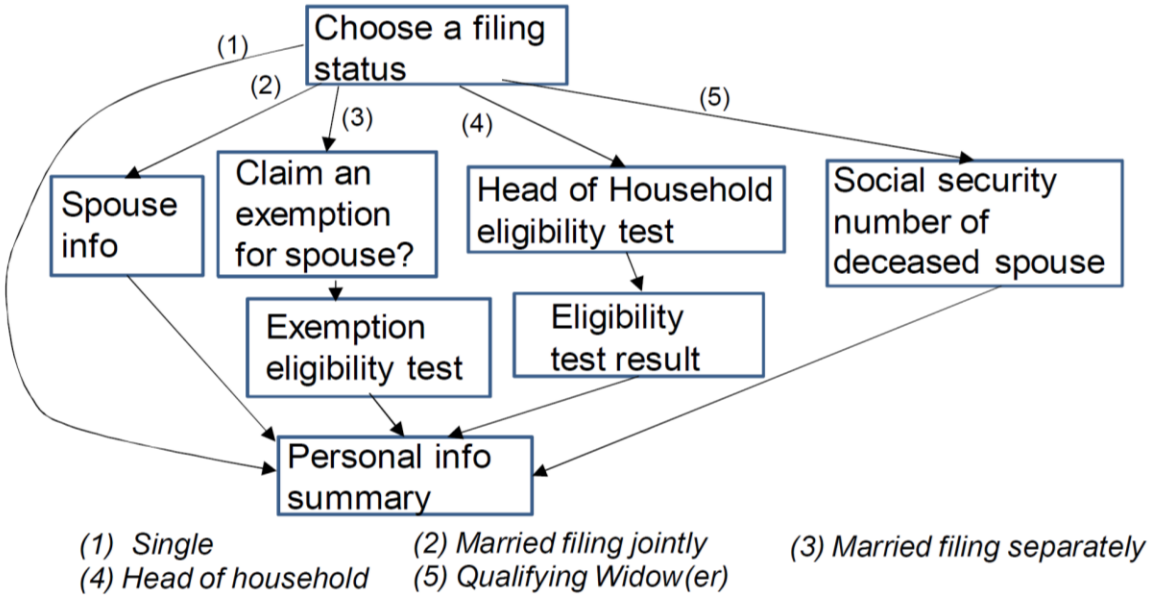
Peeking Through SSL: Traffic Analysis

- Network traffic reveals length of HTTPS packets
  - TLS supports up to 256 bytes of padding
- AJAX-rich pages have lots and lots of interactions with the server
- These interactions expose specific internal state of the page

Chen et. al., “Side-Channel Leaks in Web Applications: a Reality Today, a Challenge Tomorrow”, IEEE S&P 2010

https://spectrum.ieee.org/we-know-what-youre-watching-even-if-its-encrypted

Peeking Through SSL: An Example



Vulnerabilities in an online tax application

No easy fix. Can also be used to peek into Tor traffic

https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/WebAppSideChannel-final.pdf

Online Tools to Test SSL/TLS Security

- Qualys SSL Lab
- https://www.digicert.com/help/

Web Security Guidelines

- https://infosec.mozilla.org/guidelines/web_security
- SSL and TLS Deployment Best Practices
- https://web.dev/explore/secure